

Link for My classes:

<https://drive.google.com/drive/folders/1UUMUj2HpAZJnpjg6cX2BR5-aNRNN5Gu?usp=sharing>

SECTION 1: DOCKER

Fundamentals

1. Why was Docker created when VMs already existed?
2. What problem does containerization solve?
3. What happens internally when you run:

`docker run nginx`

4. What are Linux namespaces?
 5. What namespaces does Docker use?
 6. What are cgroups?
 7. How does Docker limit CPU and memory?
 8. Difference between containerd, runc and Docker?
 9. Explain Docker architecture end-to-end.
 10. What happens when a container starts?
-

Images

11. What is an image layer?
12. Why are Docker images immutable?
13. How does Docker layer caching work?
14. Why are some image builds slow?
15. Explain Union File System.
16. Why should Dockerfiles have fewer layers?
17. What is a dangling image?
18. What is an orphaned layer?
19. How would you reduce a 2GB image to 200MB?
20. Explain multi-stage builds internally.

Storage

21. What happens if a container crashes?
 22. Where is container data stored?
 23. Difference between volume, bind mount and tmpfs?
 24. Why are volumes preferred?
 25. How would you migrate Docker volumes between servers?
 26. Explain OverlayFS.
 27. What happens when a container writes data?
 28. Why can containers experience disk pressure?
 29. How would you troubleshoot volume corruption?
 30. What happens if Docker storage fills up?
-

Networking

31. Explain Docker networking architecture.
 32. Difference between bridge, host and overlay networks?
 33. How do containers communicate?
 34. Why can two containers on same host fail to communicate?
 35. How does Docker DNS work?
 36. Explain container-to-container communication.
 37. What happens during port mapping?
 38. How does NAT work in Docker?
 39. Why might container networking suddenly stop?
 40. How would you troubleshoot packet drops?
-

Security

41. How secure are containers compared to VMs?
42. What is rootless Docker?
43. Why is running containers as root dangerous?
44. How would you scan images for vulnerabilities?
45. How do you prevent secrets from entering images?
46. Explain image signing.
47. Explain Docker Content Trust.
48. What are seccomp profiles?
49. What are AppArmor profiles?

50. How would you harden Docker in production?

Real Production Scenarios

- 51. A container restarts every 2 minutes. How do you troubleshoot?
 - 52. A container works locally but fails in Kubernetes.
 - 53. Container memory usage continuously increases.
 - 54. Docker host disk usage suddenly hits 100%.
 - 55. Application logs disappeared after restart.
 - 56. Docker daemon crashes on production host.
 - 57. Containers cannot resolve DNS names.
 - 58. Docker image pull is extremely slow.
 - 59. Docker build takes 30 minutes. How do you optimize?
 - 60. Production container CPU spikes randomly.
-

SECTION 2: KUBERNETES

Architecture

- 61. Explain Kubernetes architecture from memory.
 - 62. What happens when `kubectl apply` is executed?
 - 63. What happens inside API Server?
 - 64. Why does Kubernetes use etcd?
 - 65. What data is stored in etcd?
 - 66. How does Scheduler choose a node?
 - 67. What happens if etcd becomes unavailable?
 - 68. What happens if API Server goes down?
 - 69. What happens if Scheduler crashes?
 - 70. What happens if Controller Manager fails?
-

Pods

- 71. Why did Kubernetes introduce Pods?
- 72. Why not manage containers directly?
- 73. Explain Pod lifecycle.

- 74. What happens when a Pod dies?
 - 75. Why do Pod IPs change?
 - 76. Difference between Pod and Container?
 - 77. Explain init containers.
 - 78. Explain sidecar containers.
 - 79. Why might a Pod remain Pending?
 - 80. Explain Pod disruption budgets.
-

Deployments

- 81. Difference between Deployment and ReplicaSet?
 - 82. What happens during rollout?
 - 83. How does rollout rollback work?
 - 84. Explain desired state.
 - 85. What happens if replicas don't match?
 - 86. Why use Deployment instead of Pod?
 - 87. How do updates happen internally?
 - 88. Explain maxUnavailable.
 - 89. Explain maxSurge.
 - 90. How would you perform zero downtime deployment?
-

Services

- 91. Why are Services required?
 - 92. Difference between Service and Ingress?
 - 93. Difference between ClusterIP, NodePort and LoadBalancer?
 - 94. How does kube-proxy work?
 - 95. How are iptables used?
 - 96. How does service discovery work?
 - 97. Explain CoreDNS.
 - 98. What happens if DNS fails?
 - 99. Why can Service exist without endpoints?
 - 100. How does traffic reach a Pod?
-

Scheduling

- 101. Explain nodeSelector.

- 102. Explain nodeAffinity.
 - 103. Explain podAffinity.
 - 104. Explain podAntiAffinity.
 - 105. Explain taints.
 - 106. Explain tolerations.
 - 107. How would you isolate workloads?
 - 108. Why are some Pods not scheduled?
 - 109. Explain topology spread constraints.
 - 110. Explain scheduler priorities.
-

Storage

- 111. Difference between PV and PVC?
 - 112. Explain StorageClass.
 - 113. What happens when PVC is created?
 - 114. Explain dynamic provisioning.
 - 115. How does CSI work?
 - 116. What happens if storage backend fails?
 - 117. How would you migrate persistent data?
 - 118. Explain ReadWriteOnce vs ReadWriteMany.
 - 119. Explain StatefulSets.
 - 120. Why are StatefulSets needed?
-

Networking

- 121. Explain Kubernetes networking model.
 - 122. How do Pods communicate across nodes?
 - 123. What is CNI?
 - 124. Explain Calico.
 - 125. Explain Cilium.
 - 126. Explain Flannel.
 - 127. How does kube-proxy route traffic?
 - 128. What happens if CNI fails?
 - 129. Explain Network Policies.
 - 130. Why can Pods communicate when Services fail?
-

Security

- 131. Explain RBAC.
 - 132. Difference between Role and ClusterRole?
 - 133. Explain Service Accounts.
 - 134. Explain Secrets.
 - 135. Why are Kubernetes Secrets not truly encrypted?
 - 136. How do you secure Secrets?
 - 137. Explain Pod Security Standards.
 - 138. Explain Admission Controllers.
 - 139. Explain OPA Gatekeeper.
 - 140. How would you harden a cluster?
-

Troubleshooting

- 141. Pod stuck in Pending.
 - 142. CrashLoopBackOff.
 - 143. ImagePullBackOff.
 - 144. OOMKilled.
 - 145. ContainerCreating forever.
 - 146. Service not reachable.
 - 147. DNS not resolving.
 - 148. Ingress not routing traffic.
 - 149. Node NotReady.
 - 150. High API Server latency.
-

Production Scenarios

- 151. One node suddenly fails. What happens?
 - 152. Entire Availability Zone fails.
 - 153. etcd database corruption.
 - 154. Cluster upgrade fails halfway.
 - 155. Massive traffic spike.
 - 156. Memory leak in production Pods.
 - 157. Application latency suddenly doubles.
 - 158. DNS outages across cluster.
 - 159. Kubernetes control plane unavailable.
 - 160. 1000 Pods start crashing simultaneously.
-

SECTION 3: TERRAFORM

Core Concepts

- 161. Explain Terraform architecture.
 - 162. What is Terraform State?
 - 163. Why is state required?
 - 164. What happens if state is deleted?
 - 165. Explain providers.
 - 166. Explain resources.
 - 167. Explain data sources.
 - 168. Explain modules.
 - 169. Explain outputs.
 - 170. Explain variables.
-

State Management

- 171. Local vs Remote state.
 - 172. Why use remote state?
 - 173. Explain state locking.
 - 174. How does locking work?
 - 175. What happens if locking fails?
 - 176. Explain backend.
 - 177. Why use Azure Storage/S3?
 - 178. How do you secure state files?
 - 179. Why are state files dangerous?
 - 180. How would you recover corrupted state?
-

Dependency Management

- 181. How does Terraform determine resource order?
- 182. Explain dependency graph.
- 183. Explain implicit dependencies.
- 184. Explain explicit dependencies.
- 185. When should depends_on be used?
- 186. How does Terraform parallelize?
- 187. What happens during plan?
- 188. What happens during apply?

- 189. What happens during refresh?
 - 190. Explain drift.
-

Advanced

- 191. Explain lifecycle blocks.
 - 192. Explain create_before_destroy.
 - 193. Explain prevent_destroy.
 - 194. Explain ignore_changes.
 - 195. Explain taint.
 - 196. Explain import.
 - 197. Explain workspaces.
 - 198. Explain count vs for_each.
 - 199. Explain dynamic blocks.
 - 200. Explain remote modules.
-

CI/CD

- 201. How do you integrate Terraform with pipelines?
 - 202. How do you secure credentials?
 - 203. How do you handle approvals?
 - 204. How do you manage environments?
 - 205. How do you manage secrets?
 - 206. How do you handle state locking in CI/CD?
 - 207. How do you perform drift detection?
 - 208. How do you implement GitOps?
 - 209. How do you manage module versions?
 - 210. How do you handle rollbacks?
-

Terraform Production Scenarios

- 211. State file accidentally deleted.
- 212. Two engineers run apply simultaneously.
- 213. Resource exists but not in state.
- 214. Apply fails halfway.
- 215. Drift detected in production.
- 216. Terraform destroys critical resource accidentally.

- 217. Backend unavailable.
 - 218. Azure Storage backend corrupted.
 - 219. Terraform upgrade breaks modules.
 - 220. Large infrastructure takes 45 minutes to apply.
-

META / STAFF ENGINEER LEVEL QUESTIONS

- 221. Design a multi-region Kubernetes platform for 10,000 developers.
- 222. Design a disaster recovery strategy for EKS/AKS.
- 223. How would you run 50,000 containers?
- 224. How would you manage 500 Terraform repositories?
- 225. Design a secure multi-tenant Kubernetes platform.
- 226. How would you reduce cloud costs by 40%?
- 227. Design a GitOps platform from scratch.
- 228. Design Kubernetes for a banking application.
- 229. Design a highly available CI/CD platform.
- 230. Design infrastructure that survives a complete regional outage.